

Software Requirements Specification (SRS)

Автоматизована система бронювання «Експрес Львів–Дрогобич»

Omni-channel платформа: Telegram Bot + Web Admin Panel

Версія документа: 1.0

Дата: 2026-02-27

Статус: Draft

Зміст

- Вступ
- Загальний опис системи
- Ролі та актори системи
- Архітектура підсистем
- Модель даних (ER-діаграма)
- Use Cases (Прецеденти використання)
- Функціональні вимоги
- Нефункціональні вимоги
- Глосарій

1. Вступ

1.1 Мета документа

Цей документ є специфікацією програмних вимог (SRS) для автоматизованої системи бронювання квитків «Експрес Львів–Дрогобич». Він призначений для команди розробки, бізнес-аналітиків та замовника та описує повний перелік функціональних і нефункціональних вимог до системи.

1.2 Контекст та проблема

Наразі облік пасажирів для міжміських мікроавтобусних рейсів здійснюється вручну: через паперові блокноти та телефонні дзвінки. Це призводить до:

- Помилки при подвійному продажі місць (overbooking)
- Відсутності централізованого журналу броней
- Неможливості аналізувати завантаженість рейсів та надійність пасажирів
- Надмірного навантаження на диспетчерів

1.3 Цілі системи

Система повинна:

- Замінити ручний облік централізованою базою даних
- Надати пасажирам самостійний доступ до бронювання через Telegram Bot
- Забезпечити водіям швидкий інструмент для управління посадкою
- Надати диспетчерам і власнику Web-панель для управління розкладом, транспортом та аналітикою
- Виключити можливість overbooking завдяки транзакційним блокуванням на рівні БД

1.4 Сфера застосування

Система обслуговує маршрути **Львів → Дрогобич** та **Дрогобич → Львів**. Платформа є омніканальною: Telegram Bot (для пасажирів та водіїв) + Web Admin Panel (для диспетчерів та власника) + Core API (центральний сервіс логіки).

2. Загальний опис системи

2.1 Перспектива продукту

Система складається з трьох взаємопов'язаних компонентів:

```
graph LR;
  A[Telegram Bot] --> B[Core API / Booking Engine];
  B --> C[PostgreSQL DB];
  D[Web Admin Panel] --> B;
```

2.2 Ключові функції (Summary)

Канал	Аудиторія	Ключові функції
Telegram Bot	Пасажири	Реєстрація, пошук рейсів, бронювання, перегляд квитків, скасування
Telegram Bot	Водії	Маніфест рейсу, зміна статусу, стоячі/посилки, boarding
Web Admin Panel	Диспетчери	Генерація розкладу, ручне бронювання, управління рейсами
Web Admin Panel	Власник	Фінансова статистика, управління автопарком, ролі, аудит

3. Ролі та актори системи

Важливо: Ролі в системі реалізовані через булеві прапорці (`is_driver`, `is_admin`, `is_superuser`) у таблиці `USERS`. Одна фізична особа може поєднувати кілька ролей одночасно (наприклад, власник може також бути пасажиром).

3.1 Пасажир (Passenger)

- **Ідентифікація:** Telegram ID + номер телефону (через шеринг контакту)
- **Канал:** Telegram Bot
- **Прапорці:** жодного (`is_driver = false`, `is_admin = false`, `is_superuser = false`)
- **Можливості:**
 - Самостійно переглядати розклад рейсів
 - Бронювати сидячі місця
 - Переглядати свої активні квитки та історію
 - Скасовувати власні бронювання

3.2 Водій (Driver)

- **Ідентифікація:** Telegram ID (zareєстрований адміністратором)
- **Канал:** Telegram Bot (розширене меню «Мої рейси»)
- **Прапорці:** `is_driver = true`
- **Можливості:**
 - Переглядати маніфест свого рейсу
 - Змінювати статус рейсу (BOARDING → ACTIVE → COMPLETED)
 - Реєструвати стоячих пасажирів в 1 клік
 - Реєструвати посилки
 - Підтверджувати посадку (boarding validation)
- **Обмеження:** Водій не вводить імена чи номери телефонів клієнтів вручну під час руху

3.3 Диспетчер (Admin)

- **Канал:** Web Admin Panel (або розширене меню бота)
- **Прапорці:** `is_admin = true`
- **Можливості:**
 - Генерувати та редагувати розклад рейсів
 - Вручну вносити бронювання за телефонними дзвінками клієнтів
 - Скасовувати рейси
 - Вирішувати конфлікти бронювань

3.4 Власник / Супер-Адмін (Owner)

- **Канал:** Web Admin Panel
- **Прапорці:** `is_superuser = true`
- **Можливості:**
 - Переглядати фінансову статистику та «касу» (audit trails)
 - Додавати та редагувати транспортні засоби (VEHICLES)
 - Призначати ролі співробітникам
 - Переглядати Trust Score пасажирів та управляти блокуваннями
 - Доступ до всього функціоналу диспетчера

4. Архітектура підсистем

4.1 Підсистема 1 — Telegram Bot UI (Client & Driver Layer)

Призначення: Інтерфейс взаємодії з кінцевими користувачами через Telegram.

Технологія: Python, aiogram 3.x

Два режими роботи:

- Пасажирський режим — доступний усім зареєстрованим користувачам
- Водійський режим — доступний лише користувачам з is_driver = true

Відповідає за: відображення inline-клавіатур, обробку callback-запитів, передачу даних до Core API.

4.2 Підсистема 2 — Web Admin Panel (Management Layer)

Призначення: Веб-інтерфейс для адміністративного управління системою.

Технологія: Web-додаток, що взаємодіє з Core API через REST.

Користувачі: Диспетчери та власники.

Відповідає за: Візуалізацію розкладу, CRM для пасажирів, управління автопарком, фінансові звіти, аудит-журнали.

4.3 Підсистема 3 — Core API & Booking Engine (Service Layer)

Призначення: Центральний сервіс бізнес-логіки системи («мозок»).

Технологія: Python (FastAPI), PostgreSQL (SQLAlchemy 2.0 + asyncpg), Alembic (міграції).

Відповідає за:

- Валідацію лімітів місць з блокуванням транзакцій (SELECT ... FOR UPDATE)
- Захист від overbooking при конкурентних запитах
- Генерацію розкладу
- Запис усіх змін у PostgreSQL
- Аудит-слід (audit trail) усіх операцій

5. Модель даних (ER-діаграма)

5.1 Таблиці та їх призначення

USERS — Усі користувачі системи

Поле	Тип	Опис
id	INT PK	Первинний ключ
telegram_id	BIGINT	Nullable, Unique. Telegram ідентифікатор
phone	STRING	Unique, Index. Номер телефону
full_name	STRING	Повне ім'я
password_hash	STRING	Для автентифікації у веб-адмінці
is_active	BOOL	Чи активний обліковий запис
is_driver	BOOL	Роль: Водій
is_admin	BOOL	Роль: Диспетчер
is_superuser	BOOL	Роль: Власник/Супер-Адмін
created_at	DATETIME	Дата реєстрації

USER_STATS — Кешована статистика пасажирів

Поле	Тип	Опис
	INT PK,	Зв'язок з USERS (1:1)

user_id Поле	FK Тип	Опис
total_trips	INT	Загальна кількість поїздок
total_noshows	INT	Кількість неявок (No-Show)
trust_score_cached	INT	Кешований рейтинг надійності

VEHICLES — Транспортні засоби

Поле	Тип	Опис
id	INT PK	Первинний ключ
plate_number	STRING	Унікальний номерний знак
model	STRING	Модель транспортного засобу
total_seats	INT	Загальна кількість сидячих місць
total_standing	INT	Ліміт стоячих місць
is_active	BOOL	Чи в активній експлуатації

LOCATIONS — Зупинки/локації

Поле	Тип	Опис
id	INT PK	Первинний ключ
name	STRING	Унікальна назва, Index

TRIPS — Рейси

Поле	Тип	Опис
id	INT PK	Первинний ключ
driver_id	INT FK	Зв'язок з USERS
vehicle_id	INT FK	Зв'язок з VEHICLES
from_location_id	INT FK	Зв'язок з LOCATIONS (пункт відправлення)
to_location_id	INT FK	Зв'язок з LOCATIONS (пункт призначення)
departure_time	DATETIME	Час відправлення, Index
arrival_time	DATETIME	Очікуваний час прибуття
status	ENUM	SCHEDULED / BOARDING / ACTIVE / COMPLETED / CLOSED / CANCELLED
seats_limit_snapshot	INT	Snapshot: копія total_seats на момент створення рейсу
standing_limit_snapshot	INT	Snapshot: копія total_standing на момент створення рейсу
price_seated	DECIMAL	Ціна сидячого місця
price_standing	DECIMAL	Ціна стоячого місця

Snapshot-принцип: Поля seats_limit_snapshot та standing_limit_snapshot копіюються з транспортного засобу у момент створення рейсу.

Подальші зміни конфігурації авто не впливають на вже створені рейси.

BOOKINGS — Бронювання (центральна транзакційна таблиця)

Поле	Тип	Опис
id	INT PK	Первинний ключ
trip_id	INT FK	Зв'язок з TRIPS, Index (trip_id, status)
passenger_id	INT FK	Nullable. Зв'язок з USERS (може бути відсутнім для стоячих/посилок)
created_by_id	INT FK	Mandatory. Хто створив запис (пасажир або диспетчер)
created_at	DATETIME	Час створення
validated_by_id	INT FK	Nullable. Хто підтвердив посадку (зазвичай водій)
validated_at	DATETIME	Nullable. Час підтвердження посадки
booking_type	ENUM	SEATED / STANDING / PARCEL
source	ENUM	BOT / WEB / PHONE / INSTAGRAM / DRIVER
status	ENUM	RESERVED / PAID / BOARDED / CANCELLED / NOSHOW
passengers_count	INT	Кількість пасажирів (0 для PARCEL)
amount_paid	DECIMAL	Сума оплати
comment	STRING	Додатковий коментар

5.2 Зв'язки між таблицями

USERS	--	USER_STATS	(1:1 — кожен user має stats)
USERS	--o{	TRIPS	(1:N — водій веде рейси)
VEHICLES	--o{	TRIPS	(1:N — авто призначається на рейси)
LOCATIONS	--o{	TRIPS	(1:N — точка відправлення/призначення)
TRIPS	--o{	BOOKINGS	(1:N — рейс містить бронювання)
USERS	--o{	BOOKINGS	(1:N — пасажир, через passenger_id)
USERS	--o{	BOOKINGS	(1:N — творець запису, через created_by_id)
USERS	--o{	BOOKINGS	(1:N — валідатор посадки, через validated_by_id)

6. Use Cases (Прецеденти використання)

6.1 Підсистема: Telegram Bot — Пасажир

UC-P1: Реєстрація (шеринг контакту Telegram)

Актор: Пасажир (неzareєстрований)

Тригер: Користувач вперше надсилає `/start` у Telegram Bot або намагається забронювати місце без реєстрації

Основний сценарій:

- Бот відображає привітальне повідомлення та кнопку «Поділитися номером телефону»
- Користувач натискає кнопку та підтверджує шеринг контакту Telegram
- Бот отримує telegram_id та phone від Telegram API
- Core API перевіряє, чи існує запис у таблиці USERS з таким phone або telegram_id
- Якщо запис не існує:** Core API створює новий запис у USERS та початковий запис у USER_STATS (total_trips = 0, trust_score_cached = 100)
- Якщо запис існує** (створений диспетчером як "тіньовий профіль"): Core API оновлює існуючий запис, прив'язуючи telegram_id
- Бот відображає головне меню пасажирів

Альтернативний сценарій — Відмова від шеринг контакту:

- A1.1: Користувач відхиляє запит на шеринг контакту
- A1.2: Бот відображає повідомлення: «Для користування ботом необхідно надати номер телефону»
- A1.3: Бот повторно пропонує кнопку шерингу

Постумова: Пасажир ідентифікований у системі та може користуватися всіма функціями бота.

UC-P2: Пошук рейсів на обрану дату

Актор: Пасажир (зареєстрований)

Тригер: Пасажир обирає «Знайти рейс» у головному меню бота

Основний сценарій:

1. Бот пропонує вибрати напрямок: «Львів → Дрогобич» або «Дрогобич → Львів»
2. Пасажир обирає напрямок
3. Бот пропонує вибрати дату (inline-календар або кнопки «Сьогодні», «Завтра»)
4. Пасажир обирає дату
5. Core API виконує запит до таблиці `TRIPS`, фільтруючи за `from_location_id`, `to_location_id`, датою `departure_time` та статусом (`SCHEDULED` або `BOARDING`)
6. Core API для кожного рейсу обчислює `seats_available = seats_limit_snapshot - SUM(passengers_count WHERE booking_type='SEATED' AND status NOT IN ('CANCELLED', 'NOSHOW'))`
7. Бот відображає список доступних рейсів із зазначенням часу відправлення, ціни та кількості вільних місць
8. Якщо вільних місць немає, відображається «Місць немає» (рейс недоступний для вибору)

Альтернативний сценарій — Рейсів немає:

- A2.1: На обрану дату та напрямок відсутні рейси зі статусом `SCHEDULED` / `BOARDING`
- A2.2: Бот відображає: «На цю дату рейсів не знайдено. Спробуйте іншу дату»

Постумова: Пасажир бачить список доступних рейсів або повідомлення про їх відсутність.

UC-P3: Бронювання сидячого місця

Актор: Пасажир (зареєстрований)

Тригер: Пасажир обирає конкретний рейс із результатів пошуку (UC-P2)

Передумова: Рейс має статус `SCHEDULED` або `BOARDING`; у Core API відображається принаймні 1 вільне сидяче місце

Основний сценарій:

1. Бот відображає деталі рейсу (маршрут, час, ціна) та кнопку «Забронювати»
2. Пасажир підтверджує бронювання
3. Core API розпочинає транзакцію БД
4. Core API виконує `SELECT seats_limit_snapshot, SUM(passengers_count) ... FOR UPDATE` для рядка рейсу — встановлює блокування
5. Core API перевіряє: `seats_available = seats_limit_snapshot - SUM(passengers_count WHERE booking_type='SEATED' AND status IN ('RESERVED', 'PAID', 'BOARDED'))`
6. Якщо `seats_available >= 1`: Core API створює запис у `BOOKINGS` з `booking_type='SEATED'`, `source='BOT'`, `status='RESERVED'`, `created_by_id = passenger.id`, `passengers_count = 1`
7. Транзакція фіксується (commit)
8. Бот відображає підтвердження бронювання з деталями квитка

Альтернативний сценарій — Місце зайнято під час транзакції (Race Condition):

- A3.1: Після отримання блокування Core API виявляє `seats_available = 0`
- A3.2: Транзакція відкочується (rollback)
- A3.3: Бот відображає: «На жаль, останнє місце щойно було заброньоване. Спробуйте інший рейс»

Постумова: У таблиці `BOOKINGS` створено новий запис; лічильник зайнятих місць оновлено.

UC-P4: Перегляд «Моїх квитків»

Актор: Пасажир (зареєстрований)

Тригер: Пасажир обирає «Мої квитки» у головному меню

Основний сценарій:

1. Core API отримує всі записи `BOOKINGS` де `passenger_id = current_user.id`
2. Записи діляться на дві групи:
 - **Активні:** статус `RESERVED`, `PAID` або `BOARDED`; `departure_time` у майбутньому
 - **Історія:** статус `BOARDED`, `CANCELLED`, `NOSHOW`; або `departure_time` у минулому
3. Бот відображає активні квитки з можливістю «Скасувати» для кожного
4. За запитом пасажир може переглянути «Історію поїздок»

Альтернативний сценарій — Немає квитків:

- A4.1: У пасажирів відсутні бронювання
 - A4.2: Бот відображає: «У вас ще немає квитків» та пропонує «Знайти рейс»
-

UC-P5: Скасування свого бронювання

Актор: Пасажир (zareestrovaniy)

Тригер: Пасажир натискає «Скасувати» для конкретного квитка (з UC-P4)

Передумова: Бронювання має статус `RESERVED` або `PAID`; рейс ще не відправився

Основний сценарій:

1. Бот запитує підтвердження: «Скасувати бронювання на [маршрут, дата, час]?»
2. Пасажир підтверджує скасування
3. Core API змінює статус запису `BOOKINGS` на `CANCELLED`
4. Бот відображає підтвердження: «Бронювання скасовано»
5. Вивільнене місце стає доступним для інших пасажирів

Альтернативний сценарій — Рейс вже відправився:

- A5.1: `departure_time` у минулому або статус рейсу `ACTIVE / COMPLETED`
- A5.2: Бот відображає: «Скасування неможливе — рейс вже відправився»

Альтернативний сценарій — Пасажир відмовляється від скасування:

- A5.3: Пасажир натискає «Ні» у підтвердженні
- A5.4: Бот повертається до екрану «Мої квитки»; зміни не вносяться

6.2 Підсистема: Telegram Bot — Водій

UC-D1: Перегляд маніфесту поточного рейсу

Актор: Водій

Тригер: Водій відкриває меню «Мої рейси» та обирає активний рейс

Передумова: Існує рейс, де `driver_id = current_user.id` та статус `SCHEDULED`, `BOARDING` або `ACTIVE`

Основний сценарій:

1. Core API повертає список рейсів, призначених водієві
2. Водій обирає конкретний рейс
3. Core API повертає всі `BOOKINGS` для цього `trip_id` зі статусом, відмінним від `CANCELLED`
4. Бот відображає маніфест: список пасажирів (ім'я, телефон для `SEATED`), стоячих (кількість), посилок, загальна кількість

Альтернативний сценарій — Немає призначених рейсів:

- A1.1: Бот відображає: «На сьогодні рейсів не призначено»

UC-D2: Зміна статусу рейсу

Актор: Водій

Тригер: Водій натискає кнопку зміни статусу у меню рейсу

Допустимі переходи статусів: `SCHEDULED` → `BOARDING` → `ACTIVE` → `COMPLETED`

Основний сценарій:

1. Бот відображає поточний статус рейсу та кнопку наступного статусу (напр., «Розпочати посадку», «Вирушити», «Завершити рейс»)
2. Водій натискає відповідну кнопку
3. Core API оновлює поле `status` у таблиці `TRIPS`
4. Бот підтверджує зміну та оновлює меню

Альтернативний сценарій — Неприпустимий перехід:

- A2.1: Водій намагається змінити статус у зворотному напрямку або пропустити крок
- A2.2: Core API повертає помилку; бот відображає «Ця дія недоступна»

UC-D3: Швидкий продаж «Стоячого» місця

Актор: Водій

Тригер: Пасажир сідає в автобус на проміжній зупинці без попереднього бронювання

Передумова: Рейс має статус `BOARDING` або `ACTIVE`; є вільні стоячі місця

Основний сценарій:

1. Водій натискає кнопку «Додати стоячого» (1 клік)
2. Core API перевіряє: `standing_available = standing_limit_snapshot - SUM(passengers_count WHERE booking_type='STANDING' AND status IN ('RESERVED', 'PAID', 'BOARDED'))`
3. Якщо `standing_available >= 1`: Core API створює запис у `BOOKINGS` з `booking_type='STANDING'`, `source='DRIVER'`, `status='BOARDED'`, `passenger_id = NULL`, `created_by_id = driver.id`, `passengers_count = 1`

- Бот відображає підтвердження: «Стоячий пасажир додано. Стоячих: X»

Альтернативний сценарій — Ліміт стоячих вичерпано:

- A3.1: `standing_available = 0`
- A3.2: Бот відображає: «Ліміт стоячих вичерпано»

UC-D4: Швидкий запис «Посилки»

Актор: Водій

Тригер: Водій отримує посилку для перевезення

Основний сценарій:

- Водій натискає кнопку «Посилка»
- Опційно: водій може додати короткий коментар (напр., «Велика коробка, с. Нагірне»)
- Core API створює запис у `BOOKINGS` з `booking_type='PARCEL'`, `source='DRIVER'`, `status='BOARDED'`, `passengers_count = 0`, `passenger_id = NULL`
- Бот підтверджує: «Посилку записано»

Примітка: `passengers_count = 0` для посилок — вони не впливають на ліміт місць у салоні.

UC-D5: Підтвердження посадки (Boarding Validation)

Актор: Водій

Тригер: Пасажир з попереднім бронюванням з'являється на посадці

Передумова: Рейс у статусі `BOARDING`; у маніфесті є пасажир зі статусом `RESERVED` або `PAID`

Основний сценарій:

- Водій знаходить пасажирів у маніфесті (UC-D1) та натискає «Підтвердити посадку» поруч із записом
- Core API оновлює запис `BOOKINGS`: `status = 'BOARDED'`, `validated_by_id = driver.id`, `validated_at = NOW()`
- Бот оновлює маніфест — пасажир відмічений як такий, що сів

Альтернативний сценарій — Пасажир не з'явився (No-Show):

- A5.1: Після відправлення рейсу (`status = ACTIVE`) Диспетчер або автоматично система змінює статус незаписаних бронювань (`RESERVED / PAID`) на `NOSHOW`
- A5.2: Core API оновлює `USER_STATS.total_noshows` для відповідного пасажирів
- A5.3: Core API перераховує `trust_score_cached`

6.3 Підсистема: Web Admin Panel — Диспетчер / Власник

UC-A1: Генерація розкладу на день/тиждень

Актор: Диспетчер або Власник

Тригер: Адміністратор відкриває розділ «Розклад» та натискає «Створити рейси»

Основний сценарій:

- Адміністратор обирає діапазон дат (день або тиждень)
- Система пропонує шаблон розкладу (час відправлення, маршрут, транспортний засіб, водій)
- Адміністратор підтверджує або редагує параметри
- Core API для кожного рейсу:
 - Створює запис у `TRIPS`
 - Копіює `total_seats` та `total_standing` з обраного `VEHICLES` у `seats_limit_snapshot` та `standing_limit_snapshot`
 - Встановлює статус `SCHEDULED`
- Система відображає підтвердження зі списком створених рейсів

Альтернативний сценарій — Конфлікт (водій або авто вже зайняті):

- A1.1: Core API виявляє, що обраний водій або авто вже призначені на інший рейс у цей час
- A1.2: Система відображає попередження та пропонує обрати інші ресурси

UC-A2: Ручне створення бронювання (за телефонним дзвінком)

Актор: Диспетчер

Тригер: Клієнт телефонує диспетчеру та просить забронювати місце

Основний сценарій:

- Диспетчер відкриває форму ручного бронювання, обирає рейс
- Диспетчер вводить номер телефону клієнта
- Core API перевіряє наявність користувача з таким `phone` у таблиці `USERS`

4. Якщо користувач існує: Core API підтягує його дані
5. Якщо користувач не існує: Core API автоматично створює «тіньовий профіль» — запис у `USERS` з `phone`, `full_name` (введеним диспетчером), `telegram_id = NULL`, `is_active = true`
6. Core API виконує бронювання: `booking_type='SEATED'`, `source='PHONE'`, `created_by_id = dispatcher.id`, `passenger_id = found_or_created_user.id`
7. Система підтверджує бронювання

Важливо щодо аудиту: `created_by_id` містить ID диспетчера, що дозволяє відстежити, хто саме вніс запис.

Альтернативний сценарій — Немає вільних місць:

- A2.1: Core API повертає помилку `SEATS_LIMIT_EXCEEDED`
- A2.2: Система відображає: «Місць на цей рейс немає. Запропонуйте клієнту альтернативний рейс»

UC-A3: Зміна статусу рейсу на CLOSED (Фінансове закриття)

Актор: Диспетчер або Власник

Тригер: Рейс завершено (`COMPLETED`); диспетчер провів звірку каси з водієм

Передумова: Рейс має статус `COMPLETED`

Основний сценарій:

1. Адміністратор відкриває завершений рейс у Web-панелі
2. Система відображає підсумок: кількість пасажирів по типах, загальна сума `SUM(amount_paid)`, список бронювань
3. Адміністратор звіряє дані з водієм та натискає «Закрити рейс (фінансово)»
4. Core API змінює `TRIPS.status` на `CLOSED`
5. Всі бронювання зі статусом `RESERVED` автоматично переводяться в `NOSHOW`
6. Система оновлює `USER_STATS` для відповідних пасажирів

Альтернативний сценарій — Рейс не у статусі COMPLETED:

- A3.1: Кнопка «Закрити рейс» недоступна для рейсів з іншим статусом

UC-A4: Перегляд Trust Score та блокування за No-Show

Актор: Диспетчер або Власник

Тригер: Адміністратор відкриває профіль пасажирів у CRM або перевіряє список ненадійних пасажирів

Основний сценарій:

1. Система відображає картку пасажирів: `full_name`, `phone`, `total_trips`, `total_noshows`, `trust_score_cached`
2. **Формула Trust Score:** `trust_score = 100 - (total_noshows / MAX(total_trips, 1)) * 100` (або інша бізнес-логіка)
3. Адміністратор може натиснути «Заблокувати» — Core API встановлює `USERS.is_active = false`
4. Заблокований пасажир не може здійснювати нові бронювання через бот

Альтернативний сценарій — Реабілітація пасажирів:

- A4.1: Власник може розблокувати пасажирів, встановивши `is_active = true`

UC-A5: Управління автопарком

Актор: Власник

Тригер: Придбання нового мікроавтобуса або зміна конфігурації існуючого

Основний сценарій:

1. Власник відкриває розділ «Автопарк» у Web-панелі
2. Натискає «Додати транспортний засіб»
3. Заповнює форму: `plate_number`, `model`, `total_seats`, `total_standing`
4. Core API створює новий запис у таблиці `VEHICLES` з `is_active = true`
5. Транспортний засіб стає доступним для призначення на рейси

Альтернативний сценарій — Редагування існуючого авто:

- A5.1: Власник редагує `total_seats` або `total_standing`
- A5.2: Core API оновлює запис у `VEHICLES`
- A5.3: Зміни НЕ впливають на вже створені рейси — `seats_limit_snapshot` у таблиці `TRIPS` залишається незмінним (Snapshot-принцип)

7. Функціональні вимоги

7.1 Управління місткістю

ID	Вимога	Пріоритет
FR-01	Система не повинна дозволяти створення бронювання типу SEATED , якщо <code>SUM(passengers_count WHERE booking_type='SEATED' AND status IN ('RESERVED','PAID','BOARDED'))</code> дорівнює або перевищує <code>seats_limit_snapshot</code> рейсу. Транзакція має відхилитися.	Must Have
FR-02	Сидячі (SEATED) та стоячі (STANDING) місця обліковуються окремо та перевіряються відносно відповідних лімітів (<code>seats_limit_snapshot</code> та <code>standing_limit_snapshot</code>).	Must Have
FR-03	Бронювання типу PARCEL повинні мати <code>passengers_count = 0</code> і не впливати на ліміт сидячих або стоячих місць.	Must Have
FR-04	При зміні конфігурації транспортного засобу (VEHICLES) вже створені рейси (TRIPS) не змінюються — використовуються збережені значення <code>seats_limit_snapshot</code> та <code>standing_limit_snapshot</code> .	Must Have

7.2 Аудит та відстеження

ID	Вимога	Пріоритет
FR-05	Кожен запис у таблиці BOOKINGS повинен мати заповнене поле <code>created_by_id</code> (ID користувача, який ініціював створення). Для телефонних бронювань це ID диспетчера, для онлайн-бронювань через бот — ID пасажирів. Поле не може бути NULL .	Must Have
FR-06	Кожне бронювання повинно мати поле <code>source</code> з одним із значень: BOT , WEB , PHONE , INSTAGRAM , DRIVER . Це поле обов'язкове та не може бути NULL .	Must Have
FR-07	Підтвердження посадки (boarding validation) записується у поля <code>validated_by_id</code> та <code>validated_at</code> у відповідному записі BOOKINGS .	Should Have

7.3 Управління користувачами

ID	Вимога	Пріоритет
FR-08	При ручному бронюванні диспетчером, якщо номер телефону клієнта відсутній у базі даних, система автоматично створює «тіньовий профіль» у таблиці USERS з <code>telegram_id = NULL</code> .	Must Have
FR-09	Якщо пасажир з «тіньовим профілем» пізніше реєструється через Telegram Bot з тим самим номером телефону, система прив'язує <code>telegram_id</code> до існуючого запису без дублювання.	Must Have
FR-10	Блокований користувач (<code>is_active = false</code>) не може створювати нові бронювання через Telegram Bot. При спробі бронювання бот відображає повідомлення про блокування.	Must Have

7.4 Trust Score та статистика

ID	Вимога	Пріоритет
FR-11	Після фінансового закриття рейсу (CLOSED) система автоматично збільшує <code>USER_STATS.total_noshows</code> для кожного пасажира, чиє бронювання залишилося у статусі noshow .	Should Have
FR-12	Після підтвердження посадки або завершення рейсу система збільшує <code>USER_STATS.total_trips</code> для відповідних пасажирів.	Should Have
FR-13	Поле <code>trust_score_cached</code> в <code>USER_STATS</code> оновлюється після кожної зміни <code>total_trips</code> або <code>total_noshows</code> .	Should Have

7.5 Управління статусами рейсу

ID	Вимога	Пріоритет
FR-14	Допустимі переходи статусів рейсу: SCHEDULED → BOARDING → ACTIVE → COMPLETED → CLOSED . Пряме скасування: будь-який статус (крім CLOSED) → CANCELLED . Зворотні або неприпустимі переходи повинні відхилитися системою.	Must Have
FR-		Must

8. Нефункціональні вимоги

8.1 Безпека від Race Conditions (Concurrency)

ID	Вимога
NFR-01	При конкурентних запитах на бронювання останнього місця система повинна гарантувати, що місце буде продане лише одному пасажиру. Реалізація: використання <code>SELECT ... FOR UPDATE</code> у транзакції PostgreSQL для блокування рядка рейсу під час перевірки місткості.
NFR-02	Усі операції перевірки та запису бронювань повинні виконуватися в межах однієї атомарної транзакції БД (рівень ізоляції <code>READ COMMITTED</code> або вище).

8.2 Продуктивність та масштабованість

ID	Вимога
NFR-03	Запити на пошук рейсів за датою та напрямком повинні виконуватися не довше 200 мс при навантаженні до 100 одночасних користувачів. Реалізація: композитний індекс по <code>(from_location_id, to_location_id, departure_time)</code> у таблиці <code>TRIPS</code> .
NFR-04	Запити на перевірку статусу квитків пасажира повинні бути покриті індексом по <code>(passenger_id, status)</code> у таблиці <code>BOOKINGS</code> .
NFR-05	Композитний індекс <code>(trip_id, status)</code> у таблиці <code>BOOKINGS</code> для ефективного підрахунку завантаженості рейсів.

8.3 UX Водія

ID	Вимога
NFR-06	Операція «Додати стоячого пасажира» повинна виконуватися в 1 клік без введення будь-яких текстових даних. Час реакції бота: не більше 2 секунд.
NFR-07	Операція «Додати посилку» може включати опційний текстовий коментар, але коментар не є обов'язковим — пропуск коментаря не блокує операцію.
NFR-08	Маніфест рейсу повинен завантажуватися не довше 3 секунд за наявності до 50 бронювань на рейс.

8.4 Технологічний стек

ID	Вимога
NFR-09	Backend: Python 3.11+, FastAPI
NFR-10	База даних: PostgreSQL 15+
NFR-11	ORM: SQLAlchemy 2.0 з асинхронним драйвером <code>asyncpg</code>
NFR-12	Telegram Bot: aiogram 3.x
NFR-13	Міграції: Alembic

8.5 Надійність та безпека

ID	Вимога
----	--------

NFR-14	Вимога 4: Паролі адміністраторів (поле password_hash) повинні зберігатися у хешованому вигляді (bcrypt або argon2).
NFR-15	API-ендпоінти, що змінюють дані, повинні вимагати автентифікації (JWT або аналог).
NFR-16	Доступ до фінансової статистики та функцій управління ролями обмежений виключно для користувачів : is_superuser = true .

9. Глосарій

Термін	Визначення
Overoverbooking	Ситуація, коли кількість бронювань перевищує фізичну місткість транспортного засобу
Race Condition	Стан системи, при якому два або більше конкурентних запити намагаються одночасно зайняти останнє доступне місце
Snapshot	Копія значення на момент створення запису; не змінюється при оновленні джерельних даних
Тіньовий профіль	Запис користувача у users , створений диспетчером для клієнта без Telegram-акаунту; telegram_id = NULL
Trust Score	Числовий рейтинг надійності пасажирів, що враховує загальну кількість поїздок та кількість неявок
No-Show	Пасажир, який забронював місце, але не з'явився на посадку
Boarding Validation	Процес підтвердження фізичної присутності пасажирів в автобусі водієм
Маніфест рейсу	Список усіх бронювань для конкретного рейсу, що використовується водієм для управління посадкою
Audit Trail	Журнал усіх дій у системі з фіксацією виконавця та часу; реалізується через поля created_by_id , validated_by_id
CLOSED	Фінальний незворотний статус рейсу після фінансової звірки між диспетчером і водієм